smxFS™

Portable FAT File System

smxFS is a FAT file system that is media-compatible with DOS/Windows. It has small code and data footprints, making it ideal for small embedded systems. smxFS supports flash media such as USB thumb drives, CompactFlash, and SD/MMC. It provides the standard C library file API.

smxFS is a FAT file system for hard real-time embedded systems. It supports fixed and removable media, and offers drivers for the media typically used in current embedded systems, such as USB thumb drives, CompactFlash, and SD/MMC cards. It is DOS/Windows media-compatible, so that media written by *smxFS* are interchangeable with Windows and other RTOSs that support the FAT file system. *smxFS* requires minimal ROM and RAM, allowing it to be used in very small embedded systems, as well as in larger ones. *smxFS* supports FAT12/16/32 and VFAT (long file names compatible with Win32 operating systems). *smxFS* uses the standard C library file API (i.e. fopen(), fread(), etc.) that is familiar to most C programmers, and it is extended with additional functions for other needed operations. *smxFS* API functions are reentrant so that *smxFS* is safe for multitasking.

Features

- FAT 12/16/32
- Long File Names (VFAT)
- DOS/Windows Media Compatible
- Media Drivers:
  - USB Mass Storage
  - SD/MMC
  - DiskOnChip®
  - CompactFlash
  - ATA/IDE Hard Drive
  - RAM Disk
  - NAND Flash Disk
  - NOR Flash Disk
- Up to 2 Terabyte Disk Size
- 15 KB Typical Code Footprint
- 7 KB Typical Data Footprint
- Standard C Library file API
- Multitasking Support
- Source Code Included
- Integrated with SMX®

Components

smxFS has six components:

1. FS API provides the standard C library API (fopen(), fread(), fwrite(), fseek(), fclose(), etc.) to the application.
2. FS Path implements the Directory Entry and FAT table structure handler. It supports FAT12/16/32 and VFAT.
3. FS Mount implements the mount/format functions for inserted devices.
4. FS Cache implements the Cache for Data, FAT, and Directory entries.
5. FS Driver Interface integrates all of the devices into the file system via a unique interface.
6. FS Port implements the OS and Compiler related definitions, macros, and functions.

Driver Interface

Seven driver interface functions have been defined to support all kinds of removable media. By using this interface, *smxFS* does not

need to know the details of the media. It treats all devices according to the same interface. If you want to add a new device driver, just implement these functions and call sfs_devreg() to register it with *smxFS*. No changes to *smxFS* files are required.

int **DriverInit** (void)
int **DriverRelease** (void)
int **DiskOpen** (void)
int **DiskClose** (void)
int **SectorRead** (addr, startsector, num)
int **SectorWrite** (addr, startsector, num)
int **IOCtl** (command, parameter)

Porting Layer

Two files, fport.h and fport.c, contain definitions, macros, and functions to port *smxFS* to any OS, compiler, and processor. Currently, they support SMX®, Windows, and Linux. (Under Linux, you can run an emulator, which uses the hard disk for testing *smxFS*.)

Configuration

smxFS has a simple configuration file, fcfg.h, to allow setting preferences and tuning RAM and ROM requirements. Cache characteristics can be set to control RAM usage. Some options can be disabled to reduce code usage, such as FAT32, VFAT, and write support.

File System API

sfs_init ()	Initialize the File System.
sfs_exit ()	Uninitialize the File System.
sfs_devreg (dev_if, id)	Register a device driver to the system.
sfs_devunreg (id)	Un-Register a device driver from the system.
sfs_devstatus (id)	Return the current status of the device/disk (e.g. mounted).
sfs_getdev (id)	Return pointer to the device information structure.
sfs_fopen (filename, mode)	Open a file for read/write access.
sfs_fclose (filehandle)	Close an open file.
sfs_fread (buf, size, items, filehandle)	Read data from an open file.
sfs_fwrite (buf, size, items, filehandle)	Write data to an open file.
sfs_fseek (filehandle, offset, method)	Move the file pointer to the specified location.
sfs_fdelete (filename)	Delete a file.
sfs_remove (filename)	Alias for sfs_fdelete().
sfs_delmany (filelist, num)	Delete many files in one operation.
sfs_copy (src, dest)	Copy a file to the same or different volume.
sfs_move (oldname, newname)	Alias for sfs_rename().
sfs_rename (oldname, newname)	Rename a file or directory. Can move a file anywhere or a directory to the same volume.
sfs_chmod (filename, mode)	Set a file's permission settings.
sfs_timestamp (filename, datetime)	Set a file's modification timestamp.
sfs_getprop (filename, fileinfo)	Get a file's properties, including attribute, size, and timestamps.
sfs_setprop (filename, fileinfo, flag)	Set a file's attribute and/or timestamps (flag indicates which).
sfs_stat (filename, fileinfo)	Get information about the file.
sfs_filelength (filename)	Return the length of a file, in bytes.
sfs_rewind (filehandle)	Move the file pointer to the beginning of the file.
sfs_fflush (filehandle)	Flush all data associated with the file handle to the storage media.
sfs_ftell (filehandle)	Determine the current file pointer position.
sfs_truncate (filehandle)	Truncate a file at the current file pointer.
sfs_feof (filehandle)	Test for end-of-file.
sfs_findfile (filename)	Test if a file exists.
sfs_findfirst (filespec, fileinfo)	Provide information about the first instance of a file whose name matches the name specified by the *filespec* argument.
sfs_findnext (id, fileinfo)	Find the next file, if any, whose name matches the *filespec* argument in a previous call to sfs_findfirst(), and return information about it in the *fileinfo* structure.

sfs_findclose (fileinfo)	Clean up after the findfirst/findnext operation.
sfs_mkdir (path)	Create a new directory.
sfs_rmdir (path)	Remove a directory.
sfs_chdir (path)	Alias for sfs_setcwd().
sfs_getcwd (buf, maxlen)	Get the current working directory for the current task.
sfs_setcwd (path)	Change the current working directory for the current task.
sfs_partition (id, partitioninfo)	Partition the storage media.
sfs_format (id, formatinfo)	Format the storage media.
sfs_getvolname (id, name)	Gets a disk's volume name.
sfs_setvolname (id, name)	Sets a disk's volume name.
sfs_freekb (id)	Return the number of free KB on the storage media.
sfs_totalkb (id)	Return the number of total KB on the storage media.
sfs_writeprotect (id)	Determine if the media is write protected.
sfs_ioctl (id, command, par)	Perform device-specific functions.

Size and Performance

Code Size

Code size varies depending upon CPU, compiler, and optimization level.

	ARM7/9 IAR	**ColdFire** CodeWarrior
API and core files (max[1])	27.5 KB	30.0 KB
CompactFlash driver	0.5 KB	1.5 KB
MMC/SD driver	4.5 KB	7.0 KB
NAND flash driver	9.0 KB	10.0 KB
NOR flash driver	5.0 KB	7.5 KB
RAM disk driver	0.5 KB	0.5 KB
USB disk driver	32.5 KB[2]	35.0 KB[2]

Notes:

1. Unused API functions are dead-stripped by the linker, so the size used by your application is likely to be much smaller.
2. The USB disk driver code is part of *smxUSBH*. The size can vary widely depending upon which Host Controller driver you are using. The size above includes the ISP1362 driver. See the *smxUSBH* brochure for other sizes.
3. Driver sizes vary because they have porting code that varies for different hardware.

Data Size (RAM Requirement)

RAM is allocated for caching data sectors, FAT table, and directory entries. The cache size depends upon the sector size. If sector size is 512 bytes, then 1.5KB is enough for the system to run. Of course, increasing the cache size will increase performance. (Using 1.5KB cache instead of the default 22KB, performance is only 20%.)

Performance

Media	Reading	Writing
CompactFlash driver	919 KB/s	695 KB/s
MMC/SD driver (SPI[1])	996 KB/s	141 KB/s
NOR flash driver	150 KB/s	30 KB/s
USB driver (EHCI)	10556 KB/s	7211 KB/s
USB driver (OHCI)	651 KB/s	523 KB/s
USB driver (ISP176x)	7023 KB/s	3072 KB/s

MMC/SD times were measured, polling. NOR flash times are for STMicro M25P16 serial flash. USB times vary significantly depending on the controller, as the examples show. See the Performance section of the smxFS User's Guide for details. Also see the smxUSBH brochure.
[1]Limited by 26MHz SPI